

Timesheet Management App — DB & Screen Design

Design only (no application code). Target: team timesheets with geo-based first-login / last-logout capture, project + task time entries with GitHub code links, two-way sync with GitHub Projects, and SSO from your existing IMS.

1. Scope & Key Rules

1. **Day attendance:** the first successful authenticated session of a calendar day (in the user's timezone) sets `first_login_at` ; the last logout/session end of that day sets `last_logout_at` . Intermediate logins/logouts are stored as events but do not overwrite the day boundaries.
 2. **Geo capture:** every login and logout event stores latitude/longitude, accuracy, IP, device, and a resolved place label. Optional geofence check marks the event `inside_site` true/false.
 3. **Work log:** during the day users create time entries (project, task, description, GitHub code link, hours or start/end).
 4. **GitHub Projects sync:** projects and tasks (issues/draft items) are pulled from GitHub Projects v2 into local `project` / `task` tables. Edits to task description in this app are pushed back to the GitHub item body.
 5. **Auth:** IMS is the identity provider (OIDC/SAML). This app trusts IMS tokens; pages can be embedded in IMS via iframe or direct link with SSO.
-

2. Database Design

2.1 Tables

organization

column	type	notes
id	uuid PK	
name	text	
default_timezone	text	e.g. <code>America/Chicago</code>

workday_cutoff	time	optional; day rollover for night shifts
----------------	------	---

app_user

column	type	notes
id	uuid PK	
org_id	uuid FK → organization	
ims_user_id	text UNIQUE	subject claim from IMS
email	text UNIQUE	
full_name	text	
github_login	text NULL	for mapping assignees
timezone	text	overrides org default
role	enum(member , manager , admin)	authorization roles kept separate from profile if you prefer a user_role table
status	enum(active , disabled)	
created_at / updated_at	timestampz	

user_role (recommended instead of a role column, prevents privilege issues)

column	type
id	uuid PK
user_id	uuid FK → app_user
role	enum(member , manager , admin)
UNIQUE(user_id, role)	

ims_identity (if a user may have multiple IMS/IdP links)

column	type	notes
id	uuid PK	

user_id	uuid FK → app_user	
provider	text	ims , github , ...
provider_subject	text	
UNIQUE(provider, provider_subject)		

work_session (raw session, one per login)

column	type	notes
id	uuid PK	
user_id	uuid FK	
login_at	timestampz	
logout_at	timestampz NULL	null = active
logout_reason	enum(user , timeout , forced , unknown)	
login_event_id	uuid FK → geo_event	
logout_event_id	uuid FK → geo_event NULL	
session_seconds	int GENERATED	logout_at - login_at

geo_event (audit of every login/logout with location)

column	type	notes
id	uuid PK	
user_id	uuid FK	
event_type	enum(login , logout , heartbeat)	
occurred_at	timestampz	server time
client_reported_at	timestampz	
latitude	numeric(9,6) NULL	
longitude	numeric(9,6) NULL	
accuracy_m	numeric NULL	

geo_permission	enum(granted , denied , unavailable)	
place_label	text NULL	reverse-geocoded
site_id	uuid FK → work_site NULL	matched geofence
inside_site	boolean NULL	
ip_address	inet	
user_agent	text	
device_id	text NULL	

work_site (optional geofences: office, client site)

id, org_id, name, latitude, longitude, radius_m, is_active

attendance_day (derived, one row per user per day — the answer to "when did they start/end")

column	type	notes
id	uuid PK	
user_id	uuid FK	
work_date	date	user-local date
first_login_at	timestampz	set once, never overwritten
last_logout_at	timestampz NULL	updated on each logout
first_login_event_id	uuid FK → geo_event	
last_logout_event_id	uuid FK → geo_event	
total_session_seconds	int	sum of sessions that day
logged_seconds	int	sum of time entries that day

status	enum(open , closed , submitted , approved , rejected)	
UNIQUE(user_id, work_date)		upsert target

project

column	type	notes
id	uuid PK	
org_id	uuid FK	
name	text	
code	text NULL	internal billing code
source	enum(github , manual)	
is_active	boolean	
UNIQUE(org_id, name)		

github_project_link

column	type	notes
id	uuid PK	
project_id	uuid FK → project	
gh_owner	text	org/user login
gh_project_number	int	Projects v2 number
gh_project_node_id	text	GraphQL node id
default_repo	text NULL	owner/repo for new issues
last_synced_at	timestampz	
sync_cursor	text NULL	pagination/delta cursor
UNIQUE(gh_project_node_id)		

task

column	type	notes
id	uuid PK	
project_id	uuid FK → project	
title	text	
description	text	body synced with GitHub
status	text	mapped from GitHub single-select (e.g. Todo/In Progress/Done)
assignee_user_id	uuid FK → app_user NULL	
source	enum(github , manual)	
gh_item_node_id	text NULL UNIQUE	Projects v2 item id
gh_content_type	enum(issue , pull_request , draft) NULL	
gh_issue_number	int NULL	
gh_repo	text NULL	owner/repo
gh_url	text NULL	
gh_updated_at	timestampz NULL	remote version stamp
local_updated_at	timestampz	conflict detection
sync_state	enum(synced , pending_push , conflict , error)	
is_active	boolean	

time_entry

column	type	notes
id	uuid PK	
user_id	uuid FK	
work_date	date	
project_id	uuid FK	

task_id	uuid FK NULL	free-text task allowed if null
task_title_snapshot	text	keeps history if task renamed
description	text	what they did
started_at / ended_at	timestampz NULL	optional precise timing
duration_minutes	int	required
billable	boolean	
status	enum(draft , submitted , approved , rejected)	
created_at / updated_at	timestampz	
CHECK duration_minutes between 1 and 1440		

code_link (a time entry can cite several code locations)

column	type	notes
id	uuid PK	
time_entry_id	uuid FK ON DELETE CASCADE	
url	text	commit / PR / file+line permalink
link_type	enum(commit , pull_request , branch , file , compare , other)	derived from URL
repo	text NULL	parsed owner/repo
ref	text NULL	sha / PR number / branch
note	text NULL	

timesheet_period / approval (optional weekly submission)

id, user_id, period_start, period_end, status, submitted_at, approved_by, approved_at, comment

sync_log

column	type	notes
id	uuid PK	
direction	enum(pull , push)	
entity	enum(project , task)	
entity_id	uuid NULL	
gh_node_id	text NULL	
trigger	enum(webhook , schedule , manual , user_edit)	
request_payload / response_payload	jsonb	
status	enum(success , failed , skipped , conflict)	
error_message	text NULL	
created_at	timestamp tz	

audit_log: id, actor_user_id, action, entity, entity_id, before jsonb, after jsonb, ip, created_at.

2.2 Indexes

- geo_event(user_id, occurred_at desc), geo_event(event_type, occurred_at)
- attendance_day(user_id, work_date desc), attendance_day(work_date) for team day view
- time_entry(user_id, work_date), time_entry(project_id, work_date), time_entry(task_id)
- task(project_id, is_active), task(gh_item_node_id), task(sync_state)
- work_session(user_id, login_at desc), partial index where logout_at is null for active sessions

2.3 Attendance derivation logic (pseudo-rules, not code)

- On login: create `geo_event(login) + work_session`; `UPSERT attendance_day(user_id, work_date)`; set `first_login_at = LEAST(existing, now)` — in practice insert-only on conflict do nothing for that column.
- On logout (explicit, or session timeout job): close `work_session`, create `geo_event(logout)`, set `attendance_day.last_logout_at = GREATEST(existing, now)` and add session seconds.
- Nightly job: close sessions still open past cutoff with `logout_reason = 'timeout'`, using last heartbeat time; recompute `total_session_seconds` and `logged_seconds`; set day `status = closed`.
- Day boundary uses `app_user.timezone`; store all timestamps in UTC (`timestampz`) and convert for display/date bucketing.

3. GitHub Projects Integration Design

Pull (GitHub → app)

- Source: GitHub Projects v2 GraphQL API (`organization/user → projectV2 → items → content`). REST is insufficient for Projects v2.
- A `github_project_link` row per GitHub project. Scheduled pull (e.g. every 10 min) plus webhook events (`issues` , `projects_v2_item`) for near-real-time.
- Map: GitHub project → `project` ; project item (issue / draft issue / PR) → `task` . Item title → `task.title` , issue body → `task.description` , Status single-select → `task.status` , assignees → `task.assignee_user_id` via `app_user.github_login` .
- Store `gh_updated_at` and `gh_item_node_id` for idempotent upserts.

Push (app → GitHub)

- Editing `task.description` in this app marks `sync_state = pending_push` ; a worker updates the GitHub issue body (`updateIssue`) or draft item (`updateProjectV2DraftIssue`).
- Status changes push to the project's `Status` field (`updateProjectV2ItemFieldValue`).
- Append-vs-replace choice: recommended is a managed block in the issue body so human notes are preserved:

```
<!-- timesheet-app:start -->
...app-managed description...
<!-- timesheet-app:end -->
```

- **Conflict rule:** if remote `gh_updated_at` changed after our last sync and local edits are pending, set `sync_state = conflict` and surface a "remote changed" banner asking the user to keep local, keep remote, or merge. Never silently overwrite.
 - All attempts recorded in `sync_log` ; failures retried with exponential backoff, then flagged in an admin screen.
 - Auth: GitHub App installation token (preferred; per-org, fine-grained, higher rate limits) or a PAT with `project` + `repo` scopes. Store credentials in a secret store, never in the DB in plaintext.
-

4. IMS / SSO Integration Design

- **Protocol:** OIDC Authorization Code + PKCE against IMS (SAML 2.0 if that is what IMS speaks). This app is a relying party; it never stores passwords.
 - **Provisioning:** JIT-create `app_user` on first successful IMS login from claims (`sub` , `email` , `name` , groups → roles). Optional SCIM for deprovisioning.
 - **Embedding options:**
 1. *Iframe embed* — IMS renders app pages in an iframe. Requires `Content-Security-Policy: frame-ancestors <ims-origin>` , cookies `SameSite=None; Secure` , and a slim "embedded" layout with no app chrome. Geolocation in an iframe needs `allow="geolocation"` on the iframe tag.
 2. *Deep link / new tab* — simplest and most reliable for the geolocation prompt; IMS links to `/timesheet` and SSO happens silently.
 - **Session and logout:** honor IMS single logout; on IMS logout, treat as a timesheet logout event (with last known location if a fresh fix is unavailable).
 - **Geolocation dependency:** browser geolocation requires HTTPS and user permission. Define policy for denial: allow login but record `geo_permission = denied` and flag the day for manager review (do not block work).
-

5. Screen Design

Navigation (member): Today • My Timesheet • Tasks • Reports. Manager adds: Team, Approvals, Admin.

5.1 Login / SSO landing

- Single "Continue with IMS" action; no local password form.
- Location consent card explaining why location is captured and what is stored, with "Allow location" button.

- After auth: full-screen state showing *Checked in at 9:04 AM • Downtown Office (±15 m)*.

```
+-----+
|          [Logo] Team Timesheet          |
| Sign in with your IMS account to continue |
|          [ Continue with IMS ]          |
| -----|
| (i) We record your location at check-in and |
|          check-out only.          [ Allow location ] |
+-----+
```

5.2 Today (home)

Top strip: three stat tiles — **First login** 9:04 AM (Downtown Office), **Last activity/logout** —, **Logged today** 5h 30m of 8h with progress bar. A "Check out for the day" button on the right.

Below: *Today's entries* list, each row = project • task • description snippet • duration • code-link chips • edit/delete. Sticky "Add time entry" primary button.

Empty state: "No entries yet today — log your first task."

```
+-----+
| Today  Mon 10 Aug          [ + Add time entry ] [Check out] |
+-----+
| First login 9:04 AM | Last logout -- | Logged 5h30m / 8h [==] |
| Downtown Office    | active session | 3 entries                |
+-----+
| Apollo   | #142 Fix auth redirect | 2h00 | (commit a1b2c3) ...|
| Apollo   | #150 Review PR       | 1h30 | (PR #150)         ...|
| Internal | Standup + planning          | 2h00 |                  ...|
+-----+
```

5.3 Add / Edit time entry (modal or side sheet)

Fields in order: **Project** (searchable select, GitHub-synced projects grouped first) → **Task** (select filtered by project, shows #issue title, with "task not listed → enter manually" fallback) → **Description** (multiline, char counter, hint that it syncs to the GitHub task) → **Code links** (repeatable row: URL + auto-detected type badge + optional note; paste a GitHub URL and it parses repo/sha/PR) → **Duration** (either start/end time pickers or a duration stepper; quick chips 15m/30m/1h/2h) → **Billable** toggle.

Footer: `Save`, `Save and add another`, `Cancel`. If description differs from the GitHub task body, show a checkbox "Also update the task description in GitHub" (default on) plus last-synced timestamp.

5.4 My Timesheet (week view)

Week picker with arrows. Grid: rows = project/task, columns = Mon–Sun, cells = minutes (editable inline), right column = row total, bottom row = daily totals. Second panel per day shows first login / last logout and total session time vs logged time, with a variance warning when they diverge by more than a threshold. Buttons: `Submit week` (locks entries to `submitted`), export CSV.

5.5 Tasks (GitHub-synced)

Filters: project, status, assignee, sync state. Table: task, project, status pill, assignee, last synced, sync badge (`Synced` / `Pending` / `Conflict` / `Error`), link-out icon to GitHub. Row click opens a detail drawer: title, description editor, status select, GitHub metadata, related time entries, and a "Sync now" action. Conflict banner: *"This task changed in GitHub after your edit"* with `Keep mine` / `Use GitHub` / `Open diff`.

5.6 Team / Attendance (manager)

Date picker + team selector. Table: member, first login (with location chip and geofence icon), last logout, total session, logged hours, entry count, day status. Row expands into a timeline of login/logout events with a small map pin per event. Flags column highlights: location denied, outside geofence, no logout, logged $\ll$ session.

5.7 Approvals (manager)

List of submitted weeks: member, period, total hours, variance, submitted date. Detail view mirrors the week grid read-only with per-day comments; actions `Approve`, `Request changes` (comment required).

5.8 Reports

Filters: date range, project, member, billable. Outputs: hours by project (bar), hours by member (bar), daily trend (line), attendance summary table. Export CSV / XLSX.

5.9 Admin

- **Integrations:** GitHub connection status, linked projects list, add/remove link, sync frequency, manual full resync, sync error log (from `sync_log`).
- **IMS/SSO:** issuer, client id, claim mapping, allowed embed origins.
- **Work sites:** geofence CRUD with radius and map preview.

- **Users & roles, Policies:** workday length, timezone, allow-login-without-location, auto-logout timeout.

5.10 Embedded (IMS) variant

Same routes with `?embedded=1` : hide global nav and header, compact padding, no external links that break out of the frame, and a single "Open full app" affordance. Recommended embedded surfaces: **Today** and **Add time entry**.

6. Non-functional Notes

- **Privacy:** capture location only at login/logout (not continuous tracking); publish a retention policy (e.g. purge `geo_event` coordinates after 12 months, keep timestamps); show users their own location history; regional consent requirements may apply.
- **Time correctness:** server timestamps are authoritative; client-reported time stored separately to detect clock skew. All storage in UTC.
- **Offline/flaky network:** queue time entries client-side and replay; dedupe with a client-generated idempotency key on `time_entry`.
- **Rate limits:** batch GitHub GraphQL queries, respect secondary rate limits, and prefer webhooks over polling.
- **Security:** authorize every read/write server-side by `user_id` ; managers scoped to their team; roles in a separate `user_role` table; secrets in a vault; audit log for edits to submitted/approved entries.